

SLIIT
SE3082 – Parallel Computing



Lab 7
Exercise 02

Akash D
IT24101853

Exercise 02: Scatter + Send/Recv

Description: Improves memory efficiency by replacing MPI_Bcast with MPI_Scatter. Only the root allocates the full array, scattering exact chunks to each process. Partial sums are collected manually.

Implementation Code (sum_scatter.c)

```
#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>
#define N 1000000

int main(int argc, char **argv) {

    MPI_Init(&argc, &argv);

    int rank, size;

    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    MPI_Comm_size(MPI_COMM_WORLD, &size);

    int chunk_size = N / size;

    int *array = NULL;

    if (rank == 0) {
        array = (int *)malloc(N * sizeof(int));
        for (int i = 0; i < N; i++) array[i] = i + 1;
    }

    int *local_chunk = (int *)malloc(chunk_size * sizeof(int));

    double start = MPI_Wtime();

    MPI_Scatter(array, chunk_size, MPI_INT, local_chunk, chunk_size, MPI_INT, 0,
        MPI_COMM_WORLD);

    long long local_sum = 0;

    for (int i = 0; i < chunk_size; i++) local_sum += local_chunk[i];

    if (rank != 0) {

        MPI_Send(&local_sum, 1, MPI_LONG_LONG, 0, 0, MPI_COMM_WORLD);

    } else {

        long long total_sum = local_sum;

        for (int r = 1; r < size; r++) {
```

```
long long recv_sum;

MPI_Recv(&recv_sum, 1, MPI_LONG_LONG, r, 0, MPI_COMM_WORLD,
MPI_STATUS_IGNORE);

total_sum += recv_sum;

}

double elapsed = MPI_Wtime() - start;

printf("\n[Ex2 Scatter] Total sum: %lld, Time: %.4f sec\n", total_sum, elapsed);

}

if (rank == 0) free(array);

free(local_chunk);

MPI_Finalize();

return 0;

}
```

Test Output (4 processes)

[Ex2 Scatter] Total sum: 500000500000, Time: 0.0055 sec